

Convolutional Neural Networks with Pytorch

Purpose

The purpose of this assignment is to become familiar with PyTorch and to build a neural network using PyTorch for Fashion MNIST data for image classification. You will build a Convolutional Neural Network (CNN) which is the most popular approach to extract features from an image without too much computation (compared to a fully completed layer).

Objectives

Learners will be able to:

- Gain a fundamental understanding of PyTorch, including its key concepts and functionalities
- Learn the architecture and principles of CNN models
- Build a CNN using PyTorch to efficiently extract features from the Fashion MNIST dataset.

Technology Requirements

- GPU environment
- Jupyter Notebook
- Python3 (Python 3.8 and above)
- PyTorch
- Torchvision
- Numpy
- Matplotlib

Directions

Accessing ZyLabs

You will complete and submit your work through zyBooks's zyLabs. Follow the directions to correctly access the provided workspace:

1. Go to the Canvas Assignment, "**Submission: Convolutional Neural Networks with Pytorch Assignment**"
2. Click the "**Load Submission...in new window**" button.
3. Once in ZyLabs, click the **green button** in the Jupyter Notebook to get started.
4. Review the directions and resources provided in the description.
5. When ready, review the provided code and develop your work where instructed.

Assignment Directions

Architecture

We implement a Convolutional Neural Network as our model. We make use of the following layers in our model.

- a convolution layer for extracting features.
- batchnorm layer for normalizing the weights in the hidden layers.
- ReLU activation function for the non-linearity between layers.
- Finally fully connected layers in the end.

Model

We make use of the following convolutional neural network architecture for our dataset.

- convolution layer output_channels=16 kernel_size=3 stride=1 padding=1
- batch normalization layer
- ReLU activation layer
- max pool layer kernel_size=2 stride=2
- convolution layer output_channels=32 kernel_size=3 stride=1 padding=1
- batch normalization layer
- ReLU activation layer
- max pool layer kernel_size=2 stride=2
- convolution layer output_channels=64 kernel_size=5 stride=1 padding=2
- batch normalization layer
- ReLU activation layer

- max pool layer kernel_size=2 stride=2
- fully connected layer - number_of_classes

Required Tasks

Build the Model

- First, define a class called Model inheriting from Pytorch's nn.Module.
- In init(constructor), define all the layers that are used to build the model
- Define a forward function for a sequential model that takes in images as input and returns the predictions as output.

All the functions are available in the PyTorch package. Read the documentation/source code for a better understanding.

- Convolutional layer: <https://pytorch.org/docs/stable/nn.html#convolution-layers>
- Batchnorm layer: <https://pytorch.org/docs/stable/nn.html#normalization-layers>
- Activation ReLU:
<https://pytorch.org/docs/stable/nn.html#non-linear-activations-weighted-sum-nonlinearity>
- Max Pooling layer: <https://pytorch.org/docs/stable/nn.html#pooling-layers>
- Fully connected layer: <https://pytorch.org/docs/stable/nn.html#linear-layers>

Initialize the CNN Model

Define a loss criterion. In this assignment, use the cross-entropy loss between the predictions and ground truth to estimate the loss.

- CrossEntropyLoss - <https://pytorch.org/docs/stable/nn.html#crossentropyloss>

Define an optimization strategy to update the weights. In this assignment, we use the most commonly used Adam optimizer from the PyTorch package.

- Adam - <https://pytorch.org/docs/stable/optim.html#algorithms>

Training the Model

The training loop is set in the following way:

For every batch in the defined number of epochs

- Move the images and labels to the GPU by checking is_cuda
- Extract output by passing images through the model

- pass the output and ground truth to the loss criterion for batch loss
- clear the gradients
- backpropagate (compute gradients w.r.t the parameters) using backward()
- update the parameters with a single optimization step
- update the training loss for plots

repeat

Testing the Classification

In the testing loop, we don't update the weights. The trained model is tested for all the samples in test data to compute the accuracy and observe how well the model generalizes to the unseen data.

The testing loop is set in the following way:

For every batch in the testing data

- Put the model in the evaluation mode and turn off the gradients
- Move the images and labels to the device available
- extract output from the model for the input
- compute the prediction class by choosing the one with the maximum probability in the predictions.
- Compare the prediction classes with true classes.
- calculate accuracy
- update test_loss for plots

repeat

Submission Directions for Project Deliverables

Learners are expected to work on the assignment individually. Ideas and concepts may be discussed with peers or other sources can be referenced for assistance, but the submitted work must be entirely your own.

You must complete and submit your work through zyBooks's zyLabs to receive credit for the assignment:

1. To get started, use the provided Jupyter Notebook in your workspace.

2. All necessary datasets are already loaded into the workspace.
3. Execute your code by clicking the “**Run**” button in top menu bar.
4. When you are ready to submit your completed work, click on “**Submit for grading**” located on the bottom left from the notebook.
5. You will know you have completed the assignment when feedback appears below the notebook.
6. If needed: to resubmit the assignment in zyLabs
 - a. Edit your work in the provided workspace.
 - b. Run your code again.
 - c. Click “**Submit for grading**” again at the bottom of the screen.

Your submission score will automatically be populated from zyBooks into your course grade. However, the course team will review submissions after the due date has passed to ensure grades are accurate.

Evaluation

This assignment is auto-graded. There are a total of four (4) test cases. Each test case has points assigned to it. Please review the notebook to see the points assigned for each test case. A percentage score will be passed to Canvas based on your score.